



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Никола Јоловић

**Колаборативна *Juruter* биљежница са
подршком за уређивање и извршавање
у реалном времену**

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	Број:
	ЗАДАТАК ЗА ЗАВРШНИ РАД	Датум:

(Податке уноси предметни наставник - ментор)

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Никола Јоловић	Број индекса:	SV9/2022
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	Игор Дејановић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Колаборативна <i>Jupyter</i> биљежница са подршком за уређивање и извршавање у реалном времену
--

ТЕКСТ ЗАДАТКА:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Quia ipsum suspendisse ultrices gravida. Risus commodo viverra maecenas accumsan.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: □ - Студента; □ - Ментора
--



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР:	
Идентификациони број, ИБР:	
Тип документације, ТД:	Монографска документација
Тип записа, ТЗ:	Текстуални штампани материјал
Врста рада, ВР:	Дипломски - бечелор рад
Аутор, АУ:	Никола Јоловић
Ментор, МН:	Др Игор Дејановић, редовни професор
Наслов рада, НР:	Колаборативна <i>Jupyter</i> биљежница са подршком за уређивање и извршавање у реалном времену
Језик публикације, ЈП:	српски/ћирилица
Језик извода, ЈИ:	српски/енглески
Земља публикавања, ЗП:	Република Србија
Уже географско подручје, УГП:	Војводина
Година, ГО:	2026
Издавач, ИЗ:	Ауторски репринт
Место и адреса, МА:	Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО: (поглавља/страна/ цитата/табела/слика/графика/прилога)	8/35/31/0/1/0/2
Научна област, НО:	Електротехничко и рачунарско инжењерство
Научна дисциплина, НД:	Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО:	Јирутер биљежница, сарадња у реалном времену, <i>CRDT</i> , Операционе трансформације
УДК	
Чува се, ЧУ:	У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН:	
Извод, ИЗ:	Овај рад представља студију случаја софтвера <i>Crab Collab</i> , колаборативне <i>Jupyter</i> биљежнице. <i>Crab Collab</i> омогућава да више корисника уређују један документ у реалном времену, уз одржавање конзистентног погледа за све кориснике. Архитектура са централним сервером омогућава дијелење окружења за извршавање кода и пружа гаранције потребне за синхронизацију стања. Представљено рјешење користи структуру документа типа биљежнице да комбинује приступе за разрјешавање конфликта засноване на операционим трансформацијама и фракционом индексирању ради постизања конзистентности.
Датум прихватања теме, ДП:	
Датум одбране, ДО:	01.01.2025
Чланови комисије, КО:	Председник: Др Петар Петровић, ванредни професор
	Члан: Др Марко Марковић, доцент
	Члан:
Члан, ментор:	Др Игор Дејановић, редовни професор
	Потпис ментора



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	
Author, AU :	Nikola Jolović
Mentor, MN :	Igor Dejanović, Phd., full professor
Title, TI :	A Collaborative Jupyter Notebook with real-time editing and execution support
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2026
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	8/35/31/0/1/0/2
Scientific field, SF :	Electrical and Computer Engineering
Scientific discipline, SD :	Applied computer science and informatics
Subject/Key words, S/KW :	Jupyter notebook, real-time collaboration, CRDT, Operational transformation
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad
Note, N :	
Abstract, AB :	This thesis presents a case study of <i>Crab Collab</i> , a collaborative <i>Jupyter</i> notebook. <i>Crab Collab</i> allows multiple users to edit one document in real-time, while maintaining a consistent view for all users. Architecture with a central server allows sharing a code execution environment and provides guarantees required for state synchronization. The presented solution utilizes notebook document structure to combine conflict resolution strategies based on operational transformations and fractional indexing to achieve consistency.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	01.01.2025
Defended Board, DB :	President: Petar Petrović, Phd., assoc. professor
	Member: Marko Marković, Phd., asist. professor
	Member:
Member, Mentor:	Igor Dejanović, Phd., full professor
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Мотивација	1
1.2	Циљ рада	1
1.3	Опсег рада	1
1.4	Структура рада	1
2	Теоријске основе	3
2.1	Софтвер за групни рад у реалном времену	3
2.2	<i>Jupyter</i>	6
3	Постојећа рјешења	9
3.1	<i>Google Colab</i>	9
3.2	<i>JupyterLab</i> у реалном времену	9
3.3	<i>CoCalc</i>	10
3.4	<i>Google Docs</i>	10
3.5	<i>Figma</i>	11
3.6	Закључак поглавља	11
4	Функционални и нефункционални захтјеви	13
5	Архитектура система	15
6	Имплементација	17
7	Ограничења и правци даљег развога	19
8	Закључак	21
	Списак слика	23
	Списак листинга	25
	Списак коришћених скраћеница	27
	Списак коришћених појмова	29
	Биографија	31
	Литература	33
	TODOs	35

1.1 Мотивација

Рачунарска биљежница (енг. *computational notebook*) је тип дигиталног документа који омогућава комбинавање форматираног текста, изворног кода и резултата извршавања у једном интерактивном документу [1]. Овај концепт остварују алати попут *Jupyter*-а, *R Markdown*-а и *Mathematica* биљежница. У остатку рада, појам рачунарска биљежница подразумева компатибилност са *Jupyter* стандардима.

Jupyter је пројекат који дефинише стандарде и развија алате за рад са рачунарским биљежницама [2]. *Jupyter* биљежница је дизајнирана да олакша документовање, дијелење и репродуковање анализе података [3]. То је чини широко распрострањеном у области науке о подацима [4], као и у образовању [5].

Сарадња у реалном времену подразумева да више корисника на потенцијално удаљеним локацијама уређује исти дигитални садржај и одмах види измјене које уносе остали корисници. Примјери софтвера који омогућавају овај начин рада су *Google Docs* [6] за уређивање текстуалних докумената и *Figma* [7] за уређивање графичких докумената. Распрострањеност ових и њима сличних алата говори о реалној потреби за оваквим приступом при уређивању свих врста дигиталних докумената.

Сарадња у реалном времену на *Jupyter* биљежницама олакшава дијелење контекста, подстиче истраживање и смањује трошкове комуникације. Упркос томе, постоје изазови који могу довести до ометања међу корисницима, што отежава уклањање грешака и омета ток рада [8]. Структура документа у коме су текст, код и стање извршавања тијесно повезани чини ове проблеме израженијим него код једноставнијих текстуалних докумената.

1.2 Циљ рада

Овај рад представља студију случаја софтвера *Crab Collab*, алата за колаборативно уређивање *Jupyter* биљежница у реалном времену. Циљ рада је да се, кроз развој система истраже механизми за рјешавање конфликта приликом истовремених измјена у циљу одржавања конзистентности погледа на документ, као и интеграција са дијелом система за извршавање кода.

1.3 Опсег рада

Систем представљен у овом раду реализован је као доказ концепта (енг. *proof-of-concept*, *PoC*), а не као систем спреман за продукцијску употребу. Разлог за то је очување фокуса на рјешавању кључних проблема подршке колаборације над рачунарском биљежницом у реалном времену, без увођења додатне комплексности која би сакрила суштину рјешења.

Ова одлука има директне импликације на архитектонске и имплементационе одлуке изнесене у наставку текста. У погледу постојаности података, скалабилности, управљања сесијама и комплетности скупа функционалности, усвојена су упрошћена рјешења довољна за демонстрацију концепта али не и стварну продукцијску примјену. Детаљна дискусија ограничења која проистичу из оваквог приступа, као и приједлози за њихово превазилажење, дати су у поглављу 7.

1.4 Структура рада

Рад је организован у осам поглавља која воде читаоца од теоријских основа потребних за разумијевање рјешења до конкретне имплементације система, закључујући са тренутним ограничењима и планом за даљи развој.

Након увода, друго поглавље нуди објашњење теоријских концепата потребних за разумијевање остатка рада, укључујући основе система за колаборацију у реалном времену, као и детаљнији увид у *Jupyter* модел и концепте. Дат је и преглед технологија коришћених за израду рјешења.

Треће поглавље анализира постојећа рјешења. Идентификоване су разлике између њих и онога што *Crab Collab* жели да постигне, али и преузете кључне идеје о архитектури и механизмима синхронизације.

Четврто поглавље пружа преглед функционалних и нефункционалних захтјева система. Наведене су и ставке које су експлицитно ван опсега.

Пето поглавље представља архитектуру система. Описане су основне компоненте и њихови међусобни односи и дат је преглед коришћених архитектуралних образаца.

Шесто поглавље се бави имплементацијом система. Објашњене су стратегије за рјешавање конфликта уз детаље имплементираних алгоритама, као и детаљи интеграције окружења за извршавање кода.

Седмо поглавље дискутује ограничења тренутног система и нуди приједлоге за њихово рјешавање, тиме приказујући јасан план даљег развоја.

Рад се завршава у осмом поглављу, у коме су процијене предности и мане представљеног рјешења и дате финалне смјернице за будући развој.

Ово поглавље описује теоријске основе неопходне за разумијевање рјешења и даје преглед коришћених технологија.

2.1 Софтвер за групни рад у реалном времену

Прије него што буду детаљно представљене конкретне технике за рјешавање конфликта, потребно је увести терминологију на којој су засноване. Овај одјељак дефинише појмове који се односе на групни рад у реалном времену, затим објашњава зашто синхрона сарадња захтијева аутоматско рјешавање конфликта и на крају нуди опис два доминантна алгоритамска приступа том проблему.

2.1.1 Групвер и групвер у реалном времену

Групвер (енг. *groupware*) је софтвер намијењен подршци рада више корисника на заједничком задатку, дијелећи приступ заједничком окружењу. Карактеристика која одваја групвер од других вишекорисничких система је обавјештавање о акцијама других корисника. Примјери вишекорисничких система који не спадају у групвер су системи за управљање базама података и оперативни системи са расподелом времена јер пружају ограничену подршку за обавјештења о акцијама. Обично се захтијева додатни упит ка систему да би се дошло до информације о акцији другог корисника [9].

Појединачна употреба групвер система назива се сесија. Сесију у сваком тренутку чини скуп учесника којима систем обезбјеђује интерфејс ка дијеленом садржају. На примјер, корисници система могу видјети синхронизован приказ података који се мијењају кроз вријеме [9].

Вријеме потребно систему да се акције корисника одразе у његовом сопственом интерфејсу назива се вријеме одзива (енг. *response time*). Вријеме потребно да промјене од стране једног корисника доспију у интерфејс других корисника назива се вријеме обавјештења (енг. *notification time*) [9].

Групвер у реалном времену је ужа категорија која подразумева да се обавјештења о корисничким акцијама брзо пропaгирају другим корисницима. Карактеришу га сљедеће особине:

- интерактивност: вријеме одзива мора бити кратко;
- рад у реалном времену: вријеме обавјештења мора бити упоредиво са временом одзива;
- дистрибуираност: учесници могу бити географски удаљени и немају приступ истој машини, него комуникација мора да се одвија преко рачунарске мреже;
- фокусираност: корисници често приступају истим подацима у исто вријеме [9].

2.1.2 Контрола конкурентности

Контрола конкурентности означава скуп техника којима се обезбјеђује исправно извршавање операција над дијеленим стањем и поред тога што се оне извршавају конкурентно, тј. без унапријед задатог редослиједа. Механизам контроле конкурентности пружа корисницима поглед на стање такав да су њихове операције извршаване једна за другом, иако се у стварности могу преклапати у времену [10].

Конфликт је стање у које се долази када двије или више конкурентних операција дјелују на исти дио дијеленог стања на начин који зависи од редослиједа њиховог извршавања, тј. када би различит редослијед примјене довео до различитог или неисправног резултата.

Механизме за контролу конкурентности можемо подијелити у двије категорије:

- песимистички, тј. засновани на превенцији конфликта и
- оптимистички, тј. засновани на разрјешавању конфликта [11].

Типичан песимистички механизам јесте закључавање (енг. *locking*). Закључавањем критичног ресурса, само један учесник може да дјелује на њега, тако да не може да настане разлика у стању за различите учеснике.

Оптимистички механизми подразумевају да се операције извршавају без рестрикција, те да се конфликти у стању детектују и ретроактивно отклоне. Овај приступ се ослања на претпоставку да до конфликта неће доћи у већини случајева, па ће потреба за примјеном самог механизма бити мања него уколико наметнемо обавезну контролу као код песимистичких приступа [11].

2.1.3 Захтјеви за сарадњу у реалном времену

Групвер системи се у зависности од временске димензије сарадње могу подијелити на синхроне и асинхроне [12]. Синхрони системи захтијевају одговор интерфејса одмах, док асинхрони системи, попут система за верзионисање, вики платформи или електронске поште, то не захтијевају, чиме је омогућено одлагање усклађивања конкурентних измјена истих подака, а често и ручни преглед и спајање измјена.

Захтјев за интерактивношћу и радом у реалном времену чини групвер системе синхроним [12]. У комбинацији са фокусираношћу сесија, намећу се специфични захтјеви за контролу конкурентности.

Интерактивност захтијева вријеме одзива довољно мало да корисницима дјелује тренутно, што имплицира употребу оптимистичких механизма контроле конкурентности.

У таквом окружењу фокусираност неизбијежно доводи до конфликта у стању, чије се елиминисање захтијева у реалном времену. Стога, конфликти морају бити ријешени аутоматски и без уочљивог кашњења [9].

За постизање ових захтјева у литератури се издвајају два доминантна алгоритамска приступа, описана у наредним одјељцима: операциона трансформација и *CRDT* (енг. *Conflict-free Replicated Data Types*) структуре података.

2.1.4 Операциона трансформација

Операциона трансформација (енг. *Operational transformation*, *OT*) представља оптимистички механизам за контролу конкурентности у групвер системима [9]. Основна идеја јесте да се примљена измјена трансформише у односу на конкурентне измјене примљене прије ње, тако да њен утицај остане исправан, иако је стање на које се примјењује промијењено у односу на оно над којим је измјена настала.

Разлог зашто је трансформација неопходна илустрован је следећим примјером. Нека дијељени документ представља 0-индексиран низ карактера и нека је његово почетно стање "абв". Нека корисник А уметне карактер "г" на позицију 0. Тада стање документа треба да буде "габв". Истовремено, корисник Б обрише карактер на позицији 1. Његова намјера је да обрише карактер "б". Међутим, уколико измјена корисника А буде примјењена прва, измјена корисника Б ће обрисати карактер "а" и резултујуће стање ће бити "Гбв", умјесто жељеног стања "Гав". Проблем се може ријешити трансформацијом операције корисника Б тако да брише карактер на позицији 2. За детаљнији преглед трансформације сложенијих комбинација измјена, погледати [13].

Да би систем заснован на *OT*-у био исправан, потребно је задовољити двије особине: очување узрочности и конвергенцију. Очување узрочности осигурава да се узрочно повезане измјене извршавају истим редоследом на свим мјестима. Конвергенција осигурава да ће, након што сви корисници приме исти скуп измјена, њихове копије документа бити идентичне [14].

Конвергенција конкурентних измјена зависи од исправности функције трансформације. Њена исправност формализована је кроз двије особине *TP1* и *TP2*. *TP1* захтијева да за двије конкурентне измјене важи да примјена резултата трансформације прве измјене у односу на другу резултује стањем еквивалентним оном које се добија примјеном резултата трансформације друге измјене у односу на прву. *TP2* захтијева

да трансформација треће конкурентне измјене у односу на низ од претходне двије даје идентичан резултат без обзира на редослијед операција у том низу.

TP2 је, међутим, неопходна искључиво у децентрализованим системима, гдје измјене могу стићи на различита мјеста различитим редослиједом. Уколико се систем ослања на централни сервер који успоставља јединствен, тотални редослијед свих измјена, довољно је задовољити само *TP1* [15]. Овај образац представља основу протокола коришћеног у *Google Docs*-у, гдје је управо ослањање на централни сервер истакнуто као разлог зашто сложеност обраде измјена не расте са бројем истовремених корисника [16].

Испуњавање *TP1* и *TP2* историјски се показало тежим него што се чини. Томе свједочи чињеница да сама изворна функција трансформације коју су предложили Елис и Гибс не задовољава ни *TP1* ни *TP2* [14]. Комплексност дизајна функција трансформације, чак и у централизованим поставкама, доноси потешкоће и захтијева пажљиво пројектовање и ригорозне доказе како би се гарантовала њихова исправност. Ово представља кључно ограничење ОТ приступа, које *CRDT* структуре, описане у наредном одјељку, рјешавају на другачији начин.

2.1.5 *CRDT* и сродни приступи

2.1.5.1 *CRDT*

CRDT (енг. *Conflict-free Replicated Data Type*) представља структуру података која у дистрибуираном систему гарантује конвергенцију свих реплика структуре стриктно самим њеним математичким својствима, а не засебно доказаним алгоритмом. Систем заснован на *CRDT* мора задовољити три захтјева:

- Свака реплика структуре података мора бити у могућности да се ажурира конкурентно и независно од осталих, без било какве координације.
- Неконзистентности стања мора аутоматски да ријеша механизам који је дио саме структуре података.
- Без обзира што у датом тренутку могу имати различито стање, све реплике морају конвергирати ка истом резултату [17].

Једноставан примјер *CRDT* структуре је скуп који подржава само додавање елемената. Пошто је додавање истог елемента идемпотентно, а унија скупова комутативна и асоцијативна, коначно стање скупа не зависи од редослиједа нити броја понављања примљених ажурирања. Свака реплика конвергира ка истом скупу простом унијом свих додатих елемената.

Гаранција коју *CRDT* структуре пружају назива се снажна коначна конзистентност (енг. *strong eventual consistency*). Реплике које су примиле исти скуп операција биће гарантовано у истом стању, без потребе за накнадним усклађивањем. Ова гаранција, међутим, обично захтијева додатне метаподатке и механизме за рјешавање конфликта код конкурентних измјена, што их чини захтјевним са становишта коришћене меморије и комплексности алгоритама. Примјери додатних података и механизма су јединствени идентификатори елемената, ознаке за обрисане елементе (енг. *tombstones*), векторски сат (енг. *vector clock*) и сл.

2.1.5.2 *CRDT* у контексту централизованих система

Постојање централног сервера пружа додатне гаранције које често омогућавају измјене алгоритама уклањањем метаподатака и комплексних механизма. На примјер, ако сервер уведе тотални поредак операција, елиминише се потреба за векторским сатовима. То омогућава поједностављену имплементацију и смањену употребу рачунарских ресурса [7].

Овакве структуре се не могу стриктно назвати *CRDT*, али су директно инспирисане њима и релевантна литература представља вриједан извор у том контексту. Она нуди провјерен, математички поткован темељ на ком се граде поједностављене структуре прилагођене потребама конкретних система [7].

2.1.5.3 Фракционо индексирање

Фракционо индексирање (енг. *fractional indexing*) је техника за одржавање редослиједа у листи, без потребе да се велики број елемената ренумерише при одређеним операцијама. Позиције елемената листе представљене су реалним бројевима, тако да је увијек могуће уметнути елемент између друга два [18].

Када се овој техници придружи детерминистичка техника разрјешавања конкурентних додјела исте позиције, добија се структура података која гарантује конвергенцију. Ово се може постићи додавањем метаподатака, чиме добијамо *CRDT*, или увођењем тоталног поретка помоћу централног сервера, чиме добијамо прагматичну структуру инспирисану *CRDT* принципима.

2.1.6 Поређење ОТ и *CRDT* приступа

Операциона трансформација и *CRDT* су алати који нам омогућују аутоматско рјешавање конфликта код софтвера за сарадњу у реалном времену. Два различита приступа нуде различите гаранције и имају различиту цијену употребе.

Код ОТ-а, конвергенција зависи од исправности функције трансформације, те је у њу потребно уложити посебан инжењерски напор за пројектовање и доказ коректности, што се историјски показало склоно грешкама. Код *CRDT*-а, конвергенција слиједи директно из математичких својстава структуре података, без потребе за засебним доказом исправности [19].

С друге стране, *CRDT* структуре уносе додатно оптерећење метаподацима неопходно за рјешавање конфликта без централног ауторитета. У поређењу с њима, ОТ операције обично користе мање рачунарских ресурса, али су уско семантички везане за конкретан тип података над којим дјелују.

Ове разлике се, међутим, смањују у централизованог архитектури. Постојање централног ауторитета поједностављује захтјеве наметнуте изворно пројектованим, децентрализованим верзијама оба приступа.

2.2 Jupyter

Овај одјељак представља *Jupyter* пројекат и његове компоненте, са посебним освртом на структуру докумената и архитектуру неопходну за разумијевање интеграције извршавања кода описане у поглављу 6.

2.2.1 Jupyter пројекат

Jupyter пројекат обухвата четири компоненте:

- формат документа,
- кернел,
- скуп корисничких апликација и
- протокол који их повезује [2].

Формат документа представља начин записа садржаја независан од конкретне апликације. Назива се биљежница [2].

Кернел (енг. *kernel*) је засебан процес који извршава код и враћа резултате. Кернел одржава стање у меморији између више извршавања кода. Максимално једна ћелија се може извршавати у датом тренутку [2].

Корисничка апликација, често названа и *Jupyter* фронтенд (енг. *front-end*), представља кориснички интерфејс за рад са биљежницама. Примјери су *Jupyter Notebook* и *JupyterLab* [2].

Кернел и корисничка апликација комуницирају протоколом који је описан у наставку [2].

2.2.2 Биљежница (документ)

Документ типа биљежнице организован је као низ ћелија. Ћелије могу бити један од два типа, зависно од њиховог садржаја:

- ћелије кода, које садрже изворни код и
- текстуалне ћелије, које садрже форматиран текст у маркдаун (енг. *markdown*) формату [2].

Ћелије кода могуће је извршити, што подразумева извршавање кода који је њихов садржај. Како је сваку појединачну ћелију могуће извршити, редослијед извршавања не зависи од редослиједа ћелија у документу. Свака ћелија чува и редни број извршавања (енг. *execution count*) који биљежи редослијед којим су ћелије извршене [2].

Извршавање ћелије са кодом производи излаз (енг. *output*) који се придружује тој ћелији. Излаз није ограничен на текстуални садржај. Може укључивати и слике, графике или *HTML* садржај [2].

2.2.3 ZeroMQ

ZeroMQ је библиотека за размјену порука која, за разлику од традиционалних система реда порука, не захтијева посредни процес (енг. *broker*) за преношење порука [20]. Редови порука држе се у самој меморији програма.

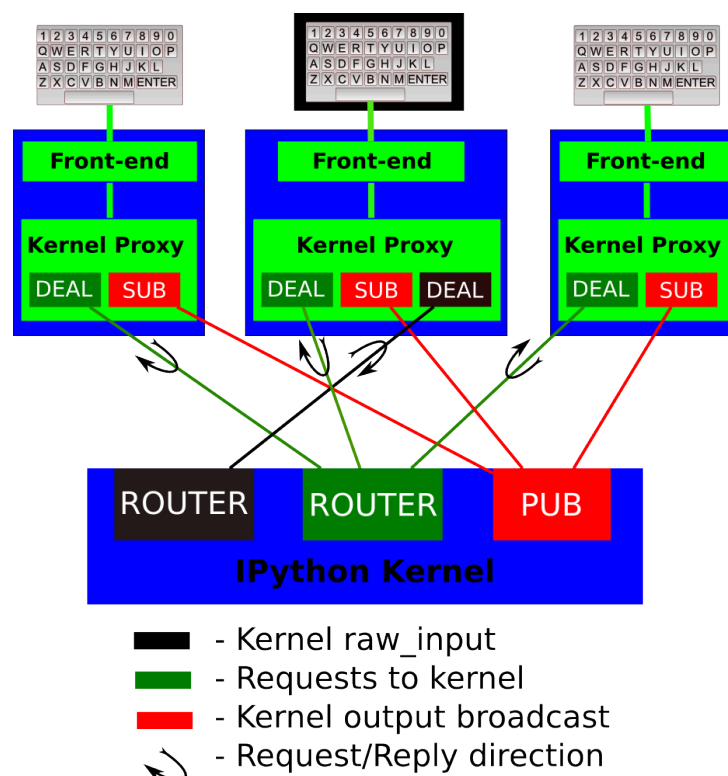
Сокет (енг. *socket*) у *ZeroMQ* је примитива за крајњу тачку комуникације. Представља асинхрону апстракцију реда порука [20]. Постоји више врста сокета и сваки подразумева засебан образац у комуникацији. Овдје је дат преглед типова сокета коришћених у *Jupyter* протоколу.

Тип захтјев-одговор (енг. *request-reply*) функционише по строгој наизмјеничној комуникацији. Једна страна шаље захтјев и очекује одговор прије него што може да пошаље сљедећи захтјев.

Тип објава-претплата (енг. *publish-subscribe*) користи истоимени образац да омогући да поруке које емитује једна страна буду примљене од стране произвољног броја претплаћених страна.

Строга наизмјеничност основног обрасца захтјев-одговор ограничава комуникацију на један неријешен захтјев у сваком тренутку. *ZeroMQ* пружа асинхронно проширење овог обрасца путем сокета типа *DEALER* и *ROUTER*, који омогућавају слање произвољног броја захтјева без чекања на одговоре.

2.2.4 Jupyter протокол



Слика 1: Архитектура комуникације између *Jupyter* фронтенда и кернела [21]¹

¹Извор: *Jupyter Development Team*. Слика је доступна под условима *Modified BSD* лиценце. Доступно на: <https://jupyter-client.readthedocs.io/en/latest/>. Приступљено: 24. августа 2026. год.

Jupyter протокол дефинише начин и формат комуникације између корисничке апликације (*Jupyter* фронтенда) и кернела [21]. Протокол је заснован на асинхроној размјени порука преко *ZeroMQ* сокета, што омогућава да се један кернел истовремено повеже са више корисничких апликација. Општа архитектура ове комуникације и распоред канала приказани су на слици 1.

Да би се спријечило блокирање комуникације у случају дуготрајних извршавања и раздвојили различити типови саобраћаја, *Jupyter* протокол користи пет засебних канала, односно *ZeroMQ* сокета [21]:

- **Shell:** *ROUTER* сокет који се користи за примарну комуникацију по обрасцу захтјев-одговор. Преко овог канала фронтенд шаље захтјеве за извршавање кода, инспекцију објеката и сл, док кернел враћа одговарајуће одговоре.
- **IOPub:** *XPUB/PUB* сокет који служи као канал за емитовање свим повезаним фронтендовима. Кернел преко овог канала објављује бочне ефекте извршавања кода (стандардни излаз, грешке, графичке приказе), као и промјене сопственог стања (нпр. да ли је заузет или слободан). На овај начин сви корисници у сесији могу видјети резултате извршавања у реалном времену.
- **Stdin:** *ROUTER* сокет који омогућава кернелу да затражи унос од корисника (нпр. при позиву Пајтон функције `input()`). Захтјев се шаље искључиво фронтенду који је покренуо извршавање, а који затим шаље кориснички унос назад кернелу преко *DEALER* сокета.
- **Control:** *ROUTER* сокет који функционише аналогно *Shell* каналу, али ради на засебној нити. Ово омогућава да се поруке високог приоритета, попут прекида извршавања (енг. *interrupt*) или гашења кернела (енг. *shutdown*), обраде тренутно без чекања да се заврши извршавање претходно започетих захтјева на *Shell* каналу.
- **Heartbeat:** Захтјев-одговор сокет који служи за контролу присутности (енг. *heartbeat*). Корисничка апликација и кернел размјењују једноставне поруке како би се тренутно детектовао евентуални губитак мрежне везе или пад кернел процеса.

Поруке које се размјењују преко ових канала логички су форматиране као *JSON* објекти и садрже сљедеће компоненте [21]:

- **header:** садржи метаподатке о самој поруци, укључујући јединствени идентификатор поруке (`msg_id`), идентификатор сесије (`session`), тип поруке (`msg_type`), верзију протокола и временску ознаку.
- **parent_header:** за поруке које су директан одговор или споредни ефекат другог захтјева, ово поље садржи заглавље изворне поруке. Ово омогућава фронтенду да тачно повеже генерисани излаз са одговарајућом ћелијом у документу.
- **metadata:** опциони *JSON* објекат са додатним контекстуалним информацијама о поруци.
- **content:** главни садржај поруке чија структура зависи од дефинисаног типа поруке (`msg_type`).
- **buffers:** опциони низ сирових бинарних бафера за ефикасан пренос већих количина података без *JSON* енкодирања.

На нивоу мрежног преноса (енг. *wire protocol*), порука се серијализује у низ *ZeroMQ* бајт-фрејмова раздвојених делимитером `<IDS|MSG>`. Ради безбједности, свака порука садржи и *HMAC* дигитални потпис (најчешће заснован на *SHA-256* хеш функцији) који се израчунава помоћу тајног кључа дефинисаног у датотеци повезивања (енг. *connection file*). Тиме се осигурава да кернел извршава само наредбе које потичу од овлашћених корисника [21].

У овом поглављу су описана постојећа рјешења која верификовано функционишу у продукционом окружењу. Укључени су примјери дијелених *Jupyter* биљежница, као и примјери из других домена који су послужили као инспирација за архитектуру и механизме коришћене у *Crab Collab*.

3.1 Google Colab

Најпознатији софтвер за онлајн дијелене *Jupyter* биљежнице је *Google Colab*. Пружа веб интерфејс за рад са биљежницама и нуди приступ једном документу за више корисника.

Упркос распрострањености употребе и самом називу софтвера, *Google Colab* има ограничену подршку за колаборацију у реалном времену. Конкурентне измјене садржаја исте ћелије од стране више корисника разрјешавају се по принципу „последњи запис побеђује” (енг. *last-write-wins*, *LWW*), који подразумева да последња измјена одређује коначно стање и „препише” преко свега што је примљено раније али га последња измјена није била свјесна.

Постоји још софтвера за дијелене биљежнице са ограниченом подршком за сарадњу у реалном времену. *Deepnote* и *Hex* рјешавају проблем конкурентних измјена песимистичким закључавањем на нивоу ћелије, што значи да никада не може више корисника да прави измјене у садржају исте ћелије [22].

3.2 JupyterLab у реалном времену

JupyterLab је апликација која пружа веб интерфејс за рад са *Jupyter* биљежницама. Архитектура софтвера укључује веб интерфејс у прегледачу (клијент), серверску компоненту на коју се клијенти прикључују и *Jupyter* кернел којим управља сервер. Сервер има улогу медијатора између клијента, са којим комуницира преко вебсокет везе, и кернела са којим комуницира преко *Jupyter* протокола.

Подршка за сарадњу у реалном времену реализована је преко проширења *jupyter-collaboration*, које има серверску и клијентску компоненту. Додаје се нова, засебна вебсокет веза између сервера и клијента преко које се врши комуникација потребна на синхронизацију стања. Више клијената може да се повеже на један сервер, што није предвиђено иницијалном поставком апликације, те је посебна инжењерска пажња морала бити усмјерена на безбједносне импликације [23].

За контролу конкурентности користе се *CRDT* структуре података. Идентификовани су различити нивои конкурентности у самој структури документа, тако да се за редослијед ћелија, као и за садржај сваке ћелије понаособ, користи засебна, одговарајућа *CRDT* структура података [24], [25].

Сваки клијент држи *CRDT* структуру документа локално, примјењује измјене на њу одмах након што настану, као и када стигну са сервера. Сервер има своју копију *CRDT* структуре документа, примјењује долазеће измјене на њу и просљеђује их свим клијентима. Дакле, сервер има улогу посредника (и тачке перзистенције и контроле приступа), а не ауторитета за синхронизацију стања [26].

Рјешење се ослања на *Yjs* екосистем, стабилну имплементацију, испробану у продукцији која нуди исправне *CRDT* имплементације [24]. То укључује *Yjs* Јаваскрипт библиотека и *pycrdt* Пајтон омотач (енг. *bindings*) за за Раст пренос библиотеке *Yjs* [27]. Ова одлука софтверу доноси стабилност и једноставност. Цијена тога је што се, упркос архитектури са централним сервером, не користе олакшице које она пружа. То имплицира већу потрошњу рачунарских ресурса него што је неопходна, као и „закључавање” у исти *CRDT* екосистем алгоритама за све нивое конкурентности стања документа (и поредак ћелије и њихов садржај).

3.3 CoCalc

CoCalc је веб платформа за колаборативно уређивање докумената и извршавање рачунарских програма. Има подршку за уређивање *Jupyter* биљежница у реалном времену, при чему се користи исти приступ као и за остале типове докумената [28].

Једино је рјешење за колаборацију на *Jupyter* биљежницама у реалном времену са јавно документованим алгоритмом [28]. Аутор идентификује три класична својства алгорита за синхронизацију:

- конвергенцију, да стање документа буде исто за све кориснике када се свачије измјене пропaгирају;
- очување узрочности, да операција коју је корисник добио прије настанка нове операције увијек буде прије ње;
- очување намјере корисника, да ефекат операције буде оно што је корисник хтио да постигне.

CoCalc испуњава прва два својства, али не и треће, допуштајући случајеве када операције имају нежељени ефекат. Ово је експлицитна одлука о дизајну која приоритизује једноставност алгоритма и имплементације у односу на гаранције коректности у сваком граничном случају [28].

За поредак ћелија и њихове метаподатке, сукоби се рјешавају употребом *LWW* принципа, док се за текстуални садржај ћелија примјењује посебан алгоритам заснован на операцијама из *diff-match-patch* (*DMP*) библиотеке, која садржи намјенске алгоритме за операције над текстом. Ради синхронизације текста, клијенти периодично израчунавају разлику између тренутног и претходно синхронизованог стања, на основу које се потом примјењује „закрпа” на стање документа [28], [29].

Одлука да се у потпуности заобиђу два стандардна приступа са *CRDT* и операционим трансформацијама у корист једноставнијег алгоритма са стриктно мањим гаранцијама исправности доноси питање употребљивости. У случајевима мале фокусираности избјегнута је комплексност рјешавања случајева који се заиста ријетко дешавају, али са већим бројем корисника и фокусиранијим обрасцима рада постоји реална потреба за већим гаранцима контроле конкурентности.

3.4 Google Docs

Google Docs представља де факто стандард за колаборативно уређивање текста у реалном времену, нудећи високу доступност, поузданост, контролу приступа и контролу конкурентности. Са прецизно имплементираним алгоритмом операционе трансформације за текст постиже колаборацију великог броја корисника без проблема са употребом рачунарских ресурса и без потешкоћа са конвергенцијом стања и очувањем намјере корисника.

Google Docs користи архитектуру са централним сервером, који је ауторитативан за стање документа и синхронизује га са клијентима [16]. Постојање централног сервера наметнуто је потребом за контролом приступа и гаранцијама за перзистентност стања, те се контрола конкурентности такође ослања на њу, чинећи систем кохезивним и робусним.

Операциона трансформација рјешава изазов измјена у истом дијелу документа без конфликта међу њима. Протокол за колаборацију осигурава да, када се више промјена дешава у исто вријеме, свака буде исправно спојена са осталима [16]. У наставку је дат преглед протокола за колаборацију на високом нивоу.

Ради постизања малог времена одзива, све измјене на клијентској страни се одмах примјењују на документ. Само једна измјена у једном тренутку се шаље на сервер, док остале постоје само локално и чекају њену потврду да би биле послате. Када са сервера стигне обавјештење о измјени другог корисника, она мора бити трансформисана у односу на локалне операције, да би могла бити исправно примијењена на локалну копију документа [16].

Сервер чува ауторитативну копију документа, као и историју операција измјене. Када нова операција стигне на сервер, она бива трансформисана у односу на све операције настале конкурентно њој, како би могла бити исправно примијењена на документ. Да би сервер знао које операције су конкурентне (настале

према истој верзији документа), прати се број ревизије и клијенти шаљу последњи број ревизије за који знају уз сваку операцију [16].

Овај протокол за колаборацију гарантује прецизност измјена и брзину, а главне предности укључују:

- ефикасност: преко мреже се шаље минимум информација потребних за опис промјене;
- константну комплексност колаборације: комплексност не расте са бројем клијената, јер сервер не познаје њихово стање;
- дистрибуираност: сервер и клијенти имају одвојене одговорности, тако да је терет алгорита распоређен између њих [16].

Приступ контроли конкурентности који користи *Google Docs* директно је инспирисао како се у овом раду третира текстуални садржај ћелија (видјети 6).

3.5 Figma

Софтвер за уређивање графичких докумената у реалном времену доноси изазове конкурентних измјена над комплексним стањем, изазване комплексношћу структуре документа. *Figma* ове изазове рјешава ослањајући се на рјешења инспирисана *CRDT* принципима, упрошћена гаранцијама које даје архитектура са централним сервером. Овај приступ има предност у односу на алгоритме засноване на операционој трансформацији јер је дефинисање исправне функције трансформације за комплексна стања са великим бројем различитих операција компликовано и подложно грешкама [7].

Конкретан примјер који се може позајмити у домен рачунарских биљежница је структура уређене листе са подршком за уметање, брисање и премјештање докумената. За рјешавање овог проблема, *Figma* користи технику фракционог индексирања описану у поглављу 2.1.5.3 [18].

У јуну 2025. *Figma* је увела функционалност слојева кода (енг. *code layers*) која захтијева синхронизацију великих текстуалних поља. За овај проблем одбацили су и *CRDT* структуру података и алгоритам операционе трансформације. *CRDT* приступ доносио је велики утршак меморије, јер чува јединствени идентификатор за сваки карактер. Операционе трансформације постају споре када постоји велики број конкурентних операција [30]. Ово је случај јер су блокови текста велики и подржано је офлајн уређивање, па повратак онлајн доводи до великог броја измјена над застарјелим документом. Компромисно рјешење које је одабрано је *Eg-walker* алгоритам из 2024. године који при рјешавању конфликта прави привремену *CRDT* структуру, а затим је одбацује, ослобађајући меморију [31].

3.6 Закључак поглавља

Преглед постојећих рјешења показује широк распон приступа проблему сарадње у реалном времену.

Алати из домена рачунарских биљежница проблем конкурентних измјена или заобилазе (закључавањем или *LWW* приступом), или га рјешавају уз компромисе (очувања намјере корисника или употребе рачунарских ресурса).

Системи изван овог домена, међутим, дали су конкретне градивне елементе за систем описан у овом раду.

Глава 4

Функционални и нефункционални захтјеви

Глава 7

Ограничења и правци даљег развога

У закључку дајте кратак преглед онога шта урађено, са освртом на проблеме који су решени, предности и мане решења и правце даљег развоја.

Списак слика

Слика 1	Архитектура комуникације између <i>Jupyter</i> фронтенда и кернела [21] ²	7
---------	--	---

²Извор: *Jupyter Development Team*. Слика је доступна под условима *Modified BSD* лиценце. Доступно на: <https://jupyter-client.readthedocs.io/en/latest/>. Приступљено: 24. августа 2026. год.

Списак листинга

Списак коришћених скраћеница

Скраћеница	Опис
НПР	TODO: Додати скраћенице

Списак коришћених појмова

Појам	Објашњење
Појам	TODO: Додати коришћене појмове

Биографија

Никола Јоловић рођен је 26. децембра 2002. године у Зворнику.

Одрастао је у Бијељини, гдје је завршио Основну школу Кнез Иво од Семберије и средњу Техничку школу Михајло Пупин. У основној и средњој школи истакао се на такмичењима из физике, математике и информатике, те је проглашаван за најбољег ђака у генерацији у оба степеника школовања.

Након завршене средње школе, одлази у Нови Сад, гдје уписује студије на Факултету техничких наука, смјер Софтверско инжењерство и информационе технологије. Током студија, истиче се, како високим оцјенама, тако и учешћем и dobrим резултатима на хакатонима.

Као студент стиче искуство из индустрије радећи стручну праксу у *Codolis*-у, бавећи се *full-stack* развојем софтвера. Након 2 године у *Codolis*-у, прелази на позицију стажисте у *JetBrains*-у, радећи на проблему скалирања *CI* агената за *TeamCity*.

- [1] S. Lau, I. Drosos, J. M. Markel, and P. J. Guo, “The Design Space of Computational Notebooks: An Analysis of 60 Systems in Academia and Industry,” in *2020 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, 2020, pp. 1–11.
- [2] T. Kluyver *et al.*, “Jupyter Notebooks – a publishing format for reproducible computational workflows,” in *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, F. Loizides and B. Schmidt, Eds., IOS Press, 2016, pp. 87–90.
- [3] H. Shen, “Interactive notebooks: Sharing the code,” *Nature*, vol. 515, no. 7525, p. 152, 2014.
- [4] A. Rule, A. Tabard, and J. D. Hollan, “Exploration and explanation in computational notebooks,” in *Proceedings of the 2018 CHI conference on human factors in computing systems*, 2018, pp. 1–12.
- [5] A. Davies, F. Hooley, P. Causey-Freeman, I. Eleftheriou, and G. Moulton, “Using interactive digital notebooks for bioscience and informatics education,” *PLOS Computational Biology*, vol. 16, no. 11, p. e1008326, 2020.
- [6] J. Day-Richter, “What’s different about the new Google Docs: Working together, even apart.” Accessed: August 22, 2026. [Online]. Доступно на https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_21.html
- [7] E. Wallace, “How Figma’s multiplayer technology works.” Accessed: August 22, 2026. [Online]. Доступно на <https://www.figma.com/blog/how-figmas-multiplayer-technology-works/>
- [8] A. Y. Wang, A. Mittal, C. Brooks, and S. Oney, “How data scientists use computational notebooks for real-time collaboration,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 1–30, 2019.
- [9] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” in *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, 1989, pp. 399–407.
- [10] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.
- [11] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.
- [12] C. A. Ellis, S. J. Gibbs, and G. L. Rein, “Groupware: some issues and experiences,” *Communications of the ACM*, vol. 34, no. 1, pp. 38–58, 1991.
- [13] J. Day-Richter, “What’s different about the new Google Docs: Conflict resolution.” Accessed: August 22, 2026. [Online]. Доступно на https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs_22.html
- [14] C. Sun and C. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in *Proceedings of the 1998 ACM Conference on Computer Supported Cooperative Work*, 1998, pp. 59–68.
- [15] A. Randolph, H. Boucheneb, A. Imine, and A. Quintero, “On Consistency of Operational Transformation Approach,” 2013.

-
- [16] J. Day-Richter, “What’s different about the new Google Docs: Making collaboration fast.” Accessed: August 22, 2026. [Online]. Доступно на <https://drive.googleblog.com/2010/09/whats-different-about-new-google-docs.html>
- [17] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in *Symposium on Self-Stabilizing Systems*, 2011, pp. 386–400.
- [18] E. Wallace, “Realtime Editing of Ordered Sequences.” Accessed: August 23, 2026. [Online]. Доступно на <https://www.figma.com/blog/realtime-editing-of-ordered-sequences/>
- [19] D. Sun, C. Sun, A. Ng, and W. Cai, “Real differences between OT and CRDT in correctness and complexity for consistency maintenance in co-editors,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, no. CSCW1, pp. 1–30, 2020.
- [20] P. Hintjens, *ZeroMQ: Messaging for Many Applications*. O’Reilly Media, 2013.
- [21] Jupyter Development Team, “Messaging in Jupyter.” Accessed: August 24, 2026. [Online]. Доступно на <http://jupyter-client.readthedocs.org/en/latest/messaging.html>
- [22] A. Y. Wang, Z. Wu, C. Brooks, and S. Oney, ““Don’t Step on My Toes”: Resolving Editing Conflicts in Real-Time Collaboration in Computational Notebooks,” in *2024 First IDE Workshop (IDE ’24)*, 2024. doi: [10.1145/3643796.3648453](https://doi.org/10.1145/3643796.3648453).
- [23] Jupyter Development Team, “Real-time collaboration without impersonation.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyterhub.readthedocs.io/en/stable/tutorial/collaboration-users.html>
- [24] Jupyter Development Team, “jupyter-collaboration: A Jupyter Server Extension Providing Support for Y Documents.” Accessed: August 25, 2026. [Online]. Доступно на <https://github.com/jupyterlab/jupyter-collaboration>
- [25] Jupyter Development Team, “jupyter_ydoc: Jupyter document structures for collaborative editing using Yjs/pycrdt.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyter-ydoc.readthedocs.io/en/latest/schema.html>
- [26] Jupyter Development Team, “Code Architecture — jupyter_collaboration documentation.” Accessed: August 25, 2026. [Online]. Доступно на <https://jupyterlab-realtime-collaboration.readthedocs.io/en/latest/developer/architecture.html>
- [27] y-crdt contributors, “pycrdt: Python bindings for Yrs.” Accessed: August 27, 2026. [Online]. Доступно на <https://pypi.org/project/pycrdt/>
- [28] W. Stein, “Collaborative Editing in CoCalc: OT, CRDT, or something else?.” Accessed: August 25, 2026. [Online]. Доступно на <https://blog.cocalc.com/2018/10/11/collaborative-editing.html>
- [29] N. Fraser, “Differential Synchronization.” Accessed: August 25, 2026. [Online]. Доступно на <https://neil.fraser.name/writing/sync/>
- [30] D. Burke and A. Kern, “Canvas, meet code: Building Figma’s code layers.” Accessed: August 25, 2026. [Online]. Доступно на <https://www.figma.com/blog/building-figmas-code-layers/>
- [31] J. Gentle and M. Kleppmann, “Collaborative Text Editing with Eg-walker: Better, Faster, Smaller,” in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys ’25)*, 2025. doi: [10.1145/3689031.3696076](https://doi.org/10.1145/3689031.3696076).

TODOs

1. [Додати скраћенице](#)
2. [Додати коришћене појмове](#)
3. [Додати скраћенице](#)
4. [Додати коришћене појмове](#)